

Sumario

1. Visao Geral do ZEHLA ML Brain	4
1.1 Principios Arquiteturais	4
1.2 Arquitetura Geral	4
2. Arquitetura Hibrida: RAG + Fine-Tuning	5
2.1 Camada 1: RAG (Retrieval-Augmented Generation)	5
2.2 Camada 2: Fine-Tuning de Modelos Especificos	6
2.3 Complementaridade RAG + Fine-Tuning	7
3. Ingestao de Conversas WhatsApp (Voice Cloning Pipeline)	8
3.1 Mecanismo de Coleta do Historico	8
3.2 Pipeline de Sanitizacao LGPD	8
3.3 Extracao de Features de Voz (Voice Fingerprinting)	9
3.4 Geracao do Perfil de Voz Estruturado	10
4. Voice Fingerprinting: Tecnicas Avancadas	10
4.1 Extracao de Sentimento por Turno	10
4.2 Analise de Topicos com LDA	11
4.3 Detecao de Estrategias de Conversao	11
4.4 Validacao do Perfil Gerado	11
5. Pipeline de Feedback e Conversao	12
5.1 Interface MLInteractionLog	12

5.2 Mecanismo de Pontuacao de Conversao	13
5.3 Feedback Ativo do Proprietario	13
6. Integracao com DNA Wizard e Insights Engine	13
6.1 Tone Thermometer: Entrada e Saida ML	13
6.2 Discount Keys: ML-Powered Pricing Intelligence	14
6.3 Insights Engine: Do Analise ao ML	14
7. Cognitive Observability e ZCC	14
7.1 Metricas Principais	15
7.2 CognitiveObservability.tsx	15
8. Model Drift Detection e Zehla Guardian	16
8.1 Deteccao de Drift	16
8.2 Niveis de Resposta do Guardian	16
9. Isolamento Multi-Tenant	17
10. Privacidade e LGPD no Contexto ML	17
10.1 Cinco Pilares de Privacidade ML	17
10.2 Pipeline de Sanitizacao de Dados	18
11. Comandos do Agente de Desenvolvimento	18
11.1 Implementacao do Feedback Loop	18
11.2 Exportacao para Fine-Tuning	18
11.3 Atualizacao do CognitiveObservability	19
11.4 Scripts ML Disponiveis	19

12. Roadmap de Implementacao

19

12.1 Fase 1: RAG Foundation (Semanas 1-4)

20

12.2 Fase 2: Voice Cloning (Semanas 5-8)

20

12.3 Fase 3: Fine-Tuning (Semanas 9-12)

21

12.4 Fase 4: Guardian (Semanas 13-16)

21

1. Visao Geral do ZEHLA ML Brain

O ZEHLA ML Brain é o sistema nervoso central da inteligência artificial da plataforma SMARTHOTEL. Enquanto o ZEHLA Brain (camada cognitiva) gerencia a lógica de atendimento, classificação de leads e respostas em tempo real, o ML Brain é responsável por garantir que esse atendimento se torne progressivamente mais personalizado, convergente e alinhado com a identidade de cada pousada. Ele não opera como um modelo estático: aprende continuamente com cada interação WhatsApp, cada resultado de reserva e cada ajuste feito pelo dono da pousada através do DNA Wizard.

A premissa fundamental é que cada pousada possui um DNA de comunicação único. O dono que trata os clientes como "amigos de férias" usa um vocabulário, ritmo de resposta e estratégia de negociação completamente diferente daquele que opera com formalidade corporativa. O ML Brain extrai, codifica e replica esse DNA para que o agente de IA atenda com a mesma naturalidade que o proprietário faria pessoalmente. Isso não significa simplesmente copiar frases: significa compreender profundamente os padrões linguísticos, estratégicos e emocionais que tornam aquela comunicação efetiva.

Este documento rege como o agente de desenvolvimento deve implementar, monitorar e iterar sobre os componentes de Machine Learning do ZEHLA. Ele cobre desde a ingestão do histórico de conversas WhatsApp (inspirado no conceito de ferramentas como wisprflow.ai) até a geração de modelos fine-tuned por pousada, passando pela pipeline de feedback de conversação, detecção de drift e privacidade LGPD.

1.1 Principios Arquiteturais

- **Multi-Tenant Isolation:** Cada pousada possui seu próprio espaço de embeddings, modelo fine-tuned e perfil de voz. Dados de tenant A nunca influenciam o modelo de tenant B.
- **Privacy-First ML:** Todo processamento de dados pessoais segue LGPD. Mensagens são anonimizadas antes da vetorização. Embeddings são criados sobre textos sanitizados, não sobre dados brutos com nomes e telefones.
- **Continuous Learning com Guardrails:** O sistema aprende com cada interação, mas possui limites claros: validação de qualidade mínima, detecção de drift e intervenção humana automática quando necessário.
- **Hybrid Intelligence:** A combinação de RAG (para memória de longo prazo e contexto em tempo real) com Fine-Tuning (para redução de custos em alta escala) garante que o sistema seja simultaneamente preciso e econômico.
- **Observable Cognition:** Cada decisão do modelo é rastreável. O ZCC (Zehla Control Center) exibe métricas em tempo real de alinhamento de tom, taxa de conversão por modelo e alertas de drift.

1.2 Arquitetura Geral

A arquitetura do ML Brain opera em três camadas interconectadas que formam um ciclo contínuo de aprendizado. A Camada de Ingestão captura e processa as conversas WhatsApp em tempo real. A

Camada de Aprendizado transforma esses dados em representações vetoriais e modelos especializados. A Camada de Aplicação injeta o conhecimento acumulado nas respostas do agente. Cada ciclo do loop gera novos dados que realimentam a Camada de Ingestão, criando um sistema que melhora progressivamente com o uso.

Camada	Função	Tecnologias
Ingestão	Captura, sanitiza e vetoriza conversas WhatsApp	BullMQ, PGVector, Zod schemas
Aprendizado	Gera embeddings, treina modelos fine-tuned	OpenAI Ada-3, LoRA, jsonl pipelines
Aplicação	Injeta contexto e tom nas respostas do agente	RAG retrieval, system prompts dinâmicos
Observabilidade	Monitora drift, alinhamento e performance	CognitiveObservability, Grafana

Tabela 1: Camadas arquiteturais do ZEHLA ML Brain

2. Arquitetura Híbrida: RAG + Fine-Tuning

A infraestrutura de Machine Learning do ZEHLA utiliza uma abordagem em duas camadas complementares. Nenhuma das duas sozinha resolveria o problema com a qualidade e eficiência necessárias: o RAG fornece memória contextual e atualização em tempo real, enquanto o Fine-Tuning oferece redução dramática de custos e especialização profunda por pousada. A combinação das duas cria um sistema que é simultaneamente flexível (adaptável a cada nova conversa), econômico (custo por token reduzido na alta escala) e personalizado (o tom de voz e as estratégias de cada pousada são internalizadas no modelo).

2.1 Camada 1: RAG (Retrieval-Augmented Generation)

A camada RAG constitui a memória de longo prazo do sistema. Quando um lead inicia uma conversa no WhatsApp, o sistema busca nos embeddings as 3 a 5 interações de maior sucesso daquela pousada específica para guiar o tom de voz e a estratégia de resposta. O processo funciona em quatro etapas sincronizadas que garantem que o contexto mais relevante seja sempre disponibilizado antes da geração da resposta.

Na primeira etapa, chamada de **Capture**, cada mensagem enviada e recebida pelo agente de WhatsApp é armazenada no banco de dados com metadados completos: tenantId, leadId, timestamp, tipo de mensagem, sentimento detectado e tag de outcome. Na segunda etapa, chamada de **Enrich**, as mensagens são enriquecidas com análise de sentimento (via LLM lightweight), extração de entidades (datas, valores, tipo de quarto) e classificação de intenção (reserva, dúvida, reclamação, cancelamento). Na terceira etapa, chamada de **Vectorize**, batches de 50 mensagens são enviados à API de embeddings (OpenAI text-embedding-3-small) após sanitização LGPD. Na quarta etapa, chamada de **Retrieve**, quando um novo lead envia uma mensagem, o sistema gera o embedding da query e busca os top-k documentos mais similares usando cosine similarity no PGVector.

Regra de Ouro RAG: Nunca injetar embeddings brutos no prompt. Sempre usar um reranker para selecionar os 3-5 documentos mais relevantes e formatar como contexto estruturado antes de enviar ao LLM.

2.1.1 Estrategia de Chunking

O chunking de conversas WhatsApp segue uma logica diferente de documentos tradicionais. Em vez de cortar por tokens fixos, o sistema agrupa mensagens em "turnos conversacionais" delimitados por janelas temporais de ate 24 horas. Cada turno e tratado como um chunk independente. Se um turno exceder 1500 tokens, ele e subdividido por topicos usando um classificador lightweight que identifica mudancas de assunto. Cada chunk recebe metadados de: (1) outcome final da conversa (BOOKED, LOST, PENDING), (2) sentimento medio do turno, (3) categorias de topico extraidas, e (4) score de sucesso calculado pela razao entre mensagens positivas e total de mensagens.

2.1.2 Modelo de Embeddings

O sistema utiliza o modelo text-embedding-3-small da OpenAI como embedding primario por oferecer o melhor custo-beneficio para textos em portugues brasileiro. Cada embedding possui 1536 dimensoes, com custo de aproximadamente US\$ 0.02 por 1M tokens. Para pousadas com planos premium, ha opcao de migrar para text-embedding-3-large (3072 dimensoes) que oferece melhor captura de nuances semanticas em linguagem coloquial e Regionalismos brasileiros. Todos os embeddings sao armazenados no PGVector (extensao do PostgreSQL) com indice IVFFlat configurado para lists=100 e probes=10, balanceando velocidade de busca e precisao.

Parametro	Configuracao	Justificativa
Modelo	text-embedding-3-small	Melhor custo-beneficio para PT-BR
Dimensoes	1536 (small) / 3072 (large)	Trade-off precision vs storage
Chunk Size	Ate 1500 tokens por turno	Preserva contexto conversacional
Janela Temporal	24h por chunk	Cobre ciclo completo de atendimento
Top-K Retrieve	3 a 5 documentos	Enriquece sem sobrecarregar prompt
Reranker	Cohere Rerank / LLM judge	Seleciona os mais relevantes
Storage	PGVector + IVFFlat index	Busca sub-50ms em ate 1M vetores

Tabela 2: Configuracao tecnica da camada RAG

2.2 Camada 2: Fine-Tuning de Modelos Específicos

A camada de Fine-Tuning e projetada para reducao de custos em alta escala. Uma vez por semana (ou sob demanda quando acumular 200+ conversas com tag success=true), o sistema compila os logs de interacoes de uma pousada em arquivos .jsonl e os envia para a API de Fine-Tuning da OpenAI. O resultado e um modelo dedicado (gpt-4o-mini fine-tuned) que internaliza o tom de voz, as estrategias de negociacao e os padroes de resposta bem-sucedidos daquela pousada. Esse modelo dedicado

requer significativamente menos tokens no prompt de sistema, reduzindo o custo por interacao em ate 60% em comparacao com o uso exclusivo de RAG.

O processo de fine-tuning segue um pipeline rigoroso de cinco etapas. Na primeira etapa, chamada de **Select**, o sistema seleciona automaticamente conversas com outcome=BOOKED e score de alinhamento de tom superior a 0.7 (medido pela cosine similarity entre a resposta do agente e o perfil de voz esperado). Na segunda etapa, chamada de **Format**, as conversas selecionadas sao transformadas no formato jsonl exigido pela API, mantendo pares de user/assistant com contexto minimamente necessario. Na terceira etapa, chamada de **Validate**, um subconjunto de 10% e separado para validacao holdout. Na quarta etapa, chamada de **Train**, o job e submetido via API. Na quinta etapa, chamada de **Evaluate**, o modelo fine-tuned e testado contra o modelo base em 50 cenarios tipicos de atendimento e so e promovido para producao se superar o base em pelo menos 15% das metricas.

2.2.1 Formato do Dataset de Treinamento

Cada entrada do dataset .jsonl segue o formato padrao da OpenAI com tres campos obrigatorios. O campo "messages" contem a sequencia completa de mensagens da conversa, incluindo o system prompt personalizado da pousada. O campo "metadata" inclui o tenantId, o outcome da conversa e o tom detectado. O sistema aplica automaticamente regras de qualidade: conversas com menos de 4 turnos sao descartadas (insuficientes para aprendizado), conversas com mensagens maiores que 2000 caracteres sao truncadas, e conversas onde o lead expressou insatisfacao sao excluidas do conjunto de treinamento positivo. O tamanho ideal do dataset por ciclo de treinamento e entre 500 e 5000 exemplos, com o sweet spot em torno de 2000 pares para pousadas com volume medio de atendimento.

Metrica	Base (GPT-4o-mini)	Fine-Tuned	Melhoria
Custo por 1K tokens (input)	US\$ 0.15	US\$ 0.15	0% (mesmo custo base)
Tokens de system prompt	~1500	~300	-80% de overhead
Custo efetivo por conversa	US\$ 0.045	US\$ 0.018	-60%
Tone Alignment Score	0.72	0.89	+24%
Taxa de Conversao (IA)	18%	26%	+44%
Tempo de Resposta (p95)	2.1s	1.4s	-33%

Tabela 3: Comparacao de performance - Modelo Base vs Fine-Tuned

2.3 Complementaridade RAG + Fine-Tuning

As duas camadas nao competem: se complementam. O RAG fornece o contexto de longo prazo e a memoria factual (disponibilidade de quartos, precos, politicas especificas de cancelamento), enquanto o fine-tuning internaliza o comportamento linguistico e a estrategia de resposta. Em producao, o fluxo e: (1) a mensagem do lead chega, (2) o sistema busca via RAG os contextos relevantes de conversas

anteriores, (3) o modelo fine-tuned da pousada gera a resposta usando tanto o prompt de sistema quanto os contextos RAG, e (4) a resposta é enviada ao WhatsApp com o tom e estilo internalizados no modelo. Este design permite que o sistema mantenha a personalização mesmo quando o fine-tuned ainda não foi treinado (usando RAG puro no início) e evolua continuamente conforme mais dados são acumulados.

3. Ingestão de Conversas WhatsApp (Voice Cloning Pipeline)

Esta seção descreve o mecanismo pelo qual o ZEHLA ML Brain extrai o perfil de voz real de um proprietário de pousada a partir do seu histórico de conversas WhatsApp. O conceito é similar ao que ferramentas como wisprflow.ai implementam: analisar padrões reais de comunicação para construir um "DNA digital" que permita à IA replicar o estilo, o tom e a estratégia de forma natural. A diferença fundamental é que o ZEHLA opera este pipeline de forma automatizada como parte do onboarding, com consentimento explícito do proprietário e em total conformidade com a LGPD.

Quando o dono da pousada conecta seu número de WhatsApp ao ZEHLA (seja via API oficial da Meta ou via bridge segura), o sistema inicia um processo de quatro fases para extrair e codificar seu perfil de comunicação. A primeira fase é a coleta, onde o histórico é importado de forma segura. A segunda fase é a sanitização, onde dados pessoais são removidos e as mensagens são normalizadas. A terceira fase é a extração de features, onde dezenas de atributos linguísticos e comportamentais são medidos. A quarta fase é a geração do perfil, onde os atributos extraídos são compilados em um documento estruturado que alimenta o system prompt e o modelo fine-tuned.

3.1 Mecanismo de Coleta do Histórico

Existem três métodos de coleta do histórico de conversas, cada um com diferentes níveis de cobertura e complexidade técnica. O método primário é a **Exportação Nativa do WhatsApp**: o proprietário acessa Configurações > Conversas > Exportar Conversa no WhatsApp Business, seleciona as conversas relevantes com clientes (que resultaram em reservas), e envia o arquivo .txt ao ZEHLA via upload seguro no painel de onboarding. Este método é o mais simples e fornece o histórico completo em formato texto com timestamp.

O método secundário é a **API Cloud da Meta**: se o proprietário já utiliza a WhatsApp Business API, o ZEHLA pode solicitar acesso às mensagens históricas via endpoint messages da Graph API (limitado a 6 meses retroativos). Este método é mais automatizado mas requer configuração prévia do WhatsApp Business Account. O método terciário é a **Bridge de Forward**: durante o onboarding, o proprietário encaminha manualmente 30 a 50 conversas representativas para o número do ZEHLA. O sistema parserifica cada conversa encaminhada e extrai os padrões. Embora menos abrangente, este método é útil como complemento e para validação rápida.

3.2 Pipeline de Sanitização LGPD

Antes de qualquer processamento, todas as mensagens passam pela pipeline de sanitizacao que garante conformidade com a LGPD e protege tanto os dados do proprietario quanto os dados dos hóspedes anteriores. Esta pipeline opera em cinco etapas sequenciais: (1) **Deteccao de PII** usando regex e NER (Named Entity Recognition) para identificar nomes, telefones, emails, CPFs, enderecos e dados bancarios, (2) **Substituicao por Placeholders** onde cada dado sensivel e substituido por tokens genericos ([NOME], [TELEFONE], [CPF], etc.), (3) **Validacao de Consentimento** onde o sistema verifica se o proprietario assinou o Termo de Autorizacao para uso de dados de comunicacao no ML, (4) **Registro de Audit Trail** onde cada operacao de sanitizacao e logada com timestamp e hash da mensagem original, e (5) **Descarte Seguro** onde a mensagem original (com dados pessoais) e descartada apos 72 horas, mantendo apenas a versao sanitizada para processamento.

3.3 Extracao de Features de Voz (Voice Fingerprinting)

O coracao do sistema de Voice Cloning e a engine de extracao de features, que analisa as mensagens sanitizadas e mede dezenas de atributos linguisticos e comportamentais que definem o estilo unico de comunicacao do proprietario. Estes atributos sao organizados em seis dimensoes principais, cada uma com multiplas sub-metricas que juntas formam o "Voice Fingerprint" completo da pousada.

Dimensao	Atributos Extraídos	Exemplo de Output
Vocabulario	Frequencia de palavras, termos unicos, regionalismos, gírias	{"vocab_diversity": 0.72, "top_terms": ["amor", "querido", "refresco"]}
Ritmo	Tamanho medio de mensagem, variacao, frequencia de multi-msg	{"avg_chars": 87, "std_chars": 42, "multi_msg_rate": 0.35}
Emoji	Tipo, frequencia, posicao no texto, substituiçoes por emoji	{"emoji_rate": 0.12, "top_emoji": ["", "", ""]}
Saudacao	Forma de abrir conversa, horarios, personalizacao	{"greeting_style": "informal_first_name", "time_aware": true}
Negociacao	Estrategia de desconto, urgencia, comparacao, follow-up	{"discount_style": "soft_comparison", "urgency_freq": 0.08}
Encerramento	Forma de fechar, CTA frequente, promessa de retorno	{"closing_cta_rate": 0.65, "next_step_freq": 0.45}

Tabela 4: Dimensoes do Voice Fingerprinting

3.3.1 Indice de Formalidade

Um dos indicadores mais importantes extraídos e o Indice de Formalidade, uma pontuacao de 0 a 1 que mede o grau de formalidade na comunicacao do proprietario. Este indice e calculado pela analise combinada de cinco fatores: (1) uso de pronome de tratamento (voce vs senhor/a, tu vs voce), (2) presenca de abreviaturas formais (Sr., Sra., Prezado/a), (3) uso de pontuacao rigorosa vs pontuacao casual (multiplos pontos de exclamacao, ausencia de virgulas), (4) vocabulario tecnico vs coloquial, e (5) uso de emojis e emoticons. Um indice de 0.0-0.3 indica estilo "Amigao de Infancia" do Tone Thermometer, 0.3-0.5 indica "Anfitriio Carinhoso", 0.5-0.7 indica "Conselheiro Local", 0.7-0.85 indica

"Gestor Eficiente", e 0.85-1.0 indica "Gerente 5 Estrelas".

3.3.2 Padroes de Resposta Temporal

Alem dos atributos textuais, o sistema extrai padroes temporais que informam o ritmo de atendimento. O **Tempo Medio de Primeira Resposta** (time-to-first-reply) e calculado a partir dos timestamps das conversas exportadas. O **Padrao de Agrupamento de Mensagens** mede se o proprietario tende a enviar mensagens longas unicas ou varias mensagens curtas em sequencia. O **Horario de Pico de Atendimento** identifica os periodos de maior atividade, permitindo que o agente de IA replique a sensacao de disponibilidade nos horarios em que o proprietario normalmente esta mais engajado. O **Padrao de Follow-up** mede o intervalo tipico entre o envio de uma proposta e o proximo contato proativo, informando a estrategia de nutricao automatica de leads pelo agente.

3.4 Geracao do Perfil de Voz Estruturado

Apos a extracao de todas as features, o sistema compila o **Voice Profile Document**, um arquivo JSON estruturado que serve como entrada primaria para o sistema de IA. Este documento e gerado automaticamente pelo pipeline e pode ser refinado manualmente pelo proprietario atraves do DNA Wizard (onde ele pode ajustar o tom, adicionar restricoes e personalizar diretrizes). O Voice Profile e composto por quatro secoes: (1) **Diretrizes de Tom** geradas a partir do Indice de Formalidade e vocabulario extraido, (2) **Padroes de Saudacao e Encerramento** baseados nas mensagens de abertura e fechamento mais frequentes, (3) **Vocabulario Preferido e Termos a Evitar** extraidos por frequencia e sentimento, e (4) **Estrategias de Negociacao** identificadas pelo padrao de discurso persuasivo do proprietario.

Integracao com DNA Wizard: O Voice Profile gerado automaticamente pela analise de conversas preenche os campos do Tone Thermometer do DNA Wizard. O proprietario pode validar, ajustar ou refinar cada parametro durante o onboarding, criando um perfil hibrido: dados reais + preferencias explicitas.

4. Voice Fingerprinting: Tecnicas Avancadas

O Voice Fingerprinting vai alem da analise superficial de vocabulario. Ele utiliza tecnicas de PLN avancadas para capturar nuances que definem a personalidade comunicativa do proprietario. Esta secao detalha os metodos de extracao, os algoritmos de classificacao e os mecanismos de validacao que garantem a fidelidade do perfil gerado.

4.1 Extracao de Sentimento por Turno

Cada turno conversacional e analisado por um modelo de sentimento lightweight (baseado no BERTimbau, modelo BERT fine-tuned para portugues brasileiro) que atribui uma pontuacao entre -1 (extremamente negativo) e +1 (extremamente positivo). Alem do sentimento geral, o sistema extrai tres dimensoes adicionais: (1) **Empatia**, medida pela frequencia de frases que demonstram

compreensao do ponto de vista do hóspede ("Entendo sua preocupacao", "Sei como e importante"), (2) **Entusiasmo**, medido por adjectives positivos, emojis e pontuacao exclamativa, e (3) **Proatividade**, medido pela frequencia com que o proprietario antecipa proximos passos e faz ofertas espontaneas. Estas tres dimensoes formam o "Triangulo Emocional" que e injetado no system prompt para guiar a tonalidade emocional do agente.

4.2 Analise de Topicos com LDA

O sistema utiliza Latent Dirichlet Allocation (LDA) para identificar os topicos mais recorrentes nas conversas do proprietario. Para pousadas, os topicos tipicos incluem: disponibilidade de datas, preco e formas de pagamento, informacoes sobre a localizacao, atrativos turisticos proximos, servicos inclusos (cafe da manha, estacionamento), politicas de cancelamento e reembolso, e atividades recomendadas. A distribuicao de topicos por pousada e usada para priorizar os topicos que o agente deve abordar proativamente. Por exemplo, se a analise revelar que 40% das conversas bem-sucedidas mencionam atividades turisticas locais, o agente sera instruido a incluir sugestoes de atividades em suas respostas de forma natural.

4.3 Detecao de Estrategias de Conversao

Uma das capacidades mais poderosas do Voice Fingerprinting e a deteccao automatica de estrategias de conversao que funcionaram no passado. O sistema analisa as conversas que resultaram em reserva (outcome=BOOKED) e identifica padroes recorrentes de comunicacao que estao correlacionados com o sucesso. Estes padroes sao classificados em quatro categorias: (1) **Ganchos de Abertura** - frases que geram engajamento imediato ("Que otimo que voce nos encontrou! Temos datas disponiveis para o feriado"), (2) **Construtores de Confianca** - elementos que reduzem a hesitacao ("Temos mais de 500 avaliacoes positivas no Booking", "Sou a Maria, estarei pessoal a recebe-lo"), (3) **Gatilhos de Urgencia Natural** - criacao de escassez sem ser agressiva ("Temos apenas 2 suites disponiveis para esse final de semana", "Essa promocao e valida ate sexta"), e (4) **Fechamento Suave** - tecnicas de conducao a reserva sem pressao ("Posso fazer uma reserva provisoria sem compromisso?", "Qual a melhor forma de pagamento para voce?").

Cada estrategia detectada recebe um **Score de Eficacia** calculado pela proporcao de vezes que ela aparece em conversas bem-sucedidas versus o total de conversas. Apenas estrategias com score acima de 0.3 e presentes em no minimo 5 conversas distintas sao incorporadas ao perfil ativo. Este filtro evita que anomalias estatisticas (uma frase que funcionou por coincidencia uma unica vez) sejam tratadas como padroes confiaveis.

4.4 Validacao do Perfil Gerado

Antes de ativar o perfil de voz para producao, o sistema executa um processo de validacao em tres etapas. A primeira etapa e a **Consistencia Interna**: o sistema verifica se os atributos extraídos sao mutuamente consistentes (por exemplo, um alto Indice de Formalidade nao deveria coexistir com alta frequencia de emojis). Inconsistencias geram warnings e acionam uma revisao manual pelo

proprietario. A segunda etapa e a **Comparacao com DNA Wizard**: se o proprietario ja preencheu o Tone Thermometer manualmente, o sistema compara as respostas do formulario com os dados extraídos das conversas. Divergencias superiores a 20% em qualquer dimensao geram um prompt de reconciliacao. A terceira etapa e o **Teste de Turing Parcial**: o sistema gera 10 respostas simuladas usando o perfil extraído e apresenta ao proprietario para que ele avalie em uma escala de 1 a 5 o quao fiel cada resposta esta ao seu estilo real. A media das avaliacoes deve ser igual ou superior a 3.5 para aprovacao.

5. Pipeline de Feedback e Conversao

Nenhuma resposta da IA deve ir para o vazio. Este e o principio fundamental do loop de feedback obrigatorio do ZEHLA ML Brain. Cada interacao gerada pelo agente e automaticamente pareada com um mecanismo de rastreamento de resultado (conversao). Quando uma reserva e fechada, a thread inteira da conversa recebe uma pontuacao positiva. Quando um lead e perdido, a conversa e marcada para analise. Quando o lead fica pendente por mais de 48 horas sem resposta, o sistema dispara um alerta de follow-up. Este loop garante que o aprendizado do ML Brain seja sempre baseado em dados reais de resultado, nao apenas em suposicoes de qualidade.

5.1 Interface MLInteractionLog

Toda interacao processada pelo LLM e registrada na interface MLInteractionLog, que armazena os dados necessarios para o treinamento continuo. Alem dos campos obrigatorios (tenantId, leadId, threadHistory, outcome, confidenceScore), o sistema registra metadados adicionais: o modelo utilizado (base ou fine-tuned), os documentos RAG injetados (referencias), o tempo de resposta, e o score de alinhamento de tom calculado em tempo real. Esta interface estende o schema basico para suportar a complexidade do aprendizado continuo.

Campo	Tipo	Descricao
tenantId	UUID	Identificador unico da pousada
leadId	UUID	Identificador unico do lead/hospede
threadHistory	Message[]	Historico completo da conversa (sanitizado)
outcome	Enum	BOOKED LOST PENDING IGNORED
confidenceScore	Float	Score de confianca do modelo (0-1)
modelUsed	String	Nome do modelo (gpt-4o-mini, ft-pousada-xyz, etc.)
ragDocumentsUsed	UUID[]	IDs dos documentos RAG injetados no prompt
toneAlignmentScore	Float	Cosine similarity com perfil de voz esperado
responseTimeMs	Integer	Tempo total de processamento em milissegundos

vectorsGenerated	Boolean	Se embeddings foram gerados nesta interacao
feedbackManual	Enum?	THUMBS_UP THUMBS_DOWN NULL (feedback do dono)
createdAt	DateTime	Timestamp da interacao

Tabela 5: Schema completo do MLInteractionLog

5.2 Mecanismo de Pontuacao de Conversao

O sistema utiliza um mecanismo de pontuacao em tres niveis para classificar a qualidade de cada interacao. O **Nivel 1 (Outcome Binary)** atribui +1 para conversas que resultaram em reserva e -1 para conversas perdidas. Este e o sinal mais forte de aprendizado. O **Nivel 2 (Tone Alignment)** mede a distancia semantica (cosine similarity) entre a resposta gerada pelo agente e as respostas tipicas do proprietario. Scores abaixo de 0.5 marcam a conversa como "fora de tom" mesmo que o outcome seja positivo. O **Nivel 3 (Lead Progression)** mede se a interacao avancou o lead no funil de vendas (de "primeiro contato" para "cotacao enviada", por exemplo). Este nivel e especialmente util para conversas pendentes que ainda nao resultaram em booking mas mostram progresso.

5.3 Feedback Ativo do Proprietario

Alem do feedback passivo (baseado em outcomes), o sistema coleta feedback ativo do proprietario atraves de dois mecanismos. O primeiro e o **Painel de Revisao de Conversas** no ZCC, onde o proprietario pode ver transcritos recentes do agente e avaliar cada resposta com thumbs up/down. O segundo e a **Correcao em Tempo Real**: quando o proprietario assume manualmente uma conversa do agente (via transferencia), o sistema registra o ponto de intervencao e analisa a diferenca entre o que o agente disse e o que o proprietario disse a seguir. Esta diferenca e um sinal de aprendizado extremamente valioso, pois revela exatamente onde o agente divergiu do comportamento esperado. Essas correcoes manuais entram com peso 3x no dataset de fine-tuning em relacao as interacoes automaticas.

6. Integracao com DNA Wizard e Insights Engine

O ML Brain nao opera isolado: ele se integra profundamente com o DNA Wizard (ferramenta de onboarding que mapeia a comunicacao da pousada) e o Insights Engine (motor de insights comportamentais). Esta integracao cria um fluxo virtuoso onde os dados coletados no onboarding alimentam o aprendizado do ML, e os insights gerados pelo ML retroalimentam o DNA Wizard com descobertas que o proprietario pode nao ter percebido conscientemente sobre seu proprio estilo de comunicacao.

6.1 Tone Thermometer: Entrada e Saida ML

O Tone Thermometer, pilar central do DNA Wizard, opera como interface entre as preferencias explicitas do proprietario e os dados implicitos extraidos pelo ML Brain. Quando o proprietario

seleciona um arquétipo (de "Amigão de Infância" a "Gerente 5 Estrelas") durante o onboarding, essa seleção define os parâmetros iniciais do system prompt do agente. Porém, o ML Brain pode ajustar dinamicamente esse arquétipo com base nos dados reais de conversação. Por exemplo, se o proprietário se classificou como "Gestor Eficiente" mas a análise de suas conversas revela um Índice de Formalidade de 0.35 (mais próximo de "Anfitrião Carinhoso"), o sistema sugere uma reconciliação e ajusta o perfil para um modelo híbrido que respeita a intenção do proprietário mas reflete seus padrões reais.

As seis dimensões do Tone Thermometer (Vocabulário, Empatia, Urgência, Formalidade, Proatividade, Humor) são continuamente calibradas pelo ML Brain. A cada 100 novas conversas processadas, o sistema recalcula os índices e, se houver desvio superior a 15% em qualquer dimensão, notifica o proprietário no ZCC com uma proposta de atualização do perfil. Este mecanismo garante que o DNA da pousada evolua naturalmente com o tempo, capturando mudanças de estilo, estratégia ou pessoal de atendimento.

6.2 Discount Keys: ML-Powered Pricing Intelligence

As Chaves de Desconto do DNA Wizard recebem inteligência adicional do ML Brain. O sistema analisa historicamente quais chaves de desconto foram mais eficazes para cada pousada e em quais contextos. Por exemplo, se a análise revela que a "Chave Temporada" (desconto para baixa temporada) tem taxa de conversão de 42% quando usada nos primeiros 3 dias de conversa, mas apenas 15% quando usada após o dia 5, o sistema ajusta automaticamente a janela de ativação da chave. O Insights Engine complementa essa análise com dados de elasticidade de preço: para cada pousada, o ML estima a curva de demanda e identifica o "sweet spot" de desconto que maximiza receita (não apenas ocupação). Esta inteligência é exposta ao agente de IA como diretrizes de negociação contextuais: "Para este lead específico, o desconto máximo sugerido é 12% com base no perfil histórico de conversões desta pousada para este tipo de quarto nesta época do ano."

6.3 Insights Engine: Do Análise ao ML

Os 12 insights do Insights Engine (Palavra-Gatilho, Hora de Ouro, Temperatura de Preço, Vazamento de Receita, etc.) são gerados a partir dos dados de ML. Cada insight é calculado por uma query sobre o banco de vetores e o log de interações MLInteractionLog. A "Palavra-Gatilho", por exemplo, é identificada pelo ML como o termo que mais frequentemente aparece em conversas que resultaram em reserva (por exemplo, "café da manhã artesanal" pode ser a palavra-gatilho de uma pousada boutique). A "Hora de Ouro" é calculada analisando os timestamps das reservas feitas via WhatsApp e identificando a janela de 2 horas com maior taxa de conversão. O Insights Engine roda diariamente como um cron job que compila os dados do MLInteractionLog das últimas 24 horas e atualiza os 12 insights no perfil da pousada, exibindo as descobertas no ZCC como cards acionáveis.

7. Cognitive Observability e ZCC

Todo modelo utilizado pelo ZEHLA deve emitir logs para a interface de Cognitive Observability do ZCC (Zehla Control Center). Este principio, chamado de "Cognitive Observability", garante que cada decisao do sistema de IA seja transparente, rastreavel e auditavel. O ZCC exibe metricas em tempo real que permitem ao proprietario e a equipe ZEHLA entender como o agente esta performando, onde estao os gargalos e quando e necessario intervir.

7.1 Metricas Principais

O dashboard de Cognitive Observabilidade exibe seis metricas fundamentais que devem ser atualizadas em tempo real (via WebSocket) e tambem disponiveis em views historicas (diaria, semanal, mensal). A primeira metrica e a **Taxa de Conversao da IA vs Humano**: quantas reservas o bot fechou versus quantas foram fechadas quando o atendimento foi transferido para um humano. Esta metrica e o indicador mais importante de saude do sistema. A segunda metrica e o **Tone Alignment Score**: a media das distancias semanticas (cosine similarity) entre as respostas do agente e o perfil de voz esperado da pousada. Scores abaixo de 0.5 por mais de 24 horas acionam um alerta automatico. A terceira metrica e o **Model Drift Index**: medida da divergencia entre as predicoes do modelo e os outcomes reais ao longo do tempo. A quarta metrica e o **Response Latency (p95)**: tempo de resposta do agente em milissegundos. A quinta metrica e o **Handoff Rate**: frequencia de transferencias de conversas do agente para humanos. A sexta metrica e o **Lead Temperature Distribution**: distribuicao dos leads nos estagios do funil (frio, morno, quente, reservado).

Metrica	Fonte de Dados	Alerta Threshold	Frequencia
Taxa de Conversao IA	MLInteractionLog.outcome	< 10% em 7 dias	Real-time + Diario
Tone Alignment Score	Embedding cosine sim	< 0.5 por 24h	Real-time
Model Drift Index	Prediction vs Outcome	> 0.3 em 7 dias	Semanal
Response Latency p95	responseTimeMs	> 3000ms	Real-time
Handoff Rate	Transfer events	> 30% em 7 dias	Diario
Lead Temperature	Lead.stage distribution	> 60% "frio" por 5 dias	Diario

Tabela 6: Metricas de Cognitive Observability

7.2 CognitiveObservability.tsx

O componente CognitiveObservability.tsx no ZCC consome estas metricas via API interna (/api/ml/metrics) e as exibe em graficos interativos. O componente deve implementar: (1) cards de resumo com valor atual, variacao percentual semanal e badge de status (verde/amarelo/vermelho), (2) grafico de linha temporal para cada metrica com comparacao semana-a-semana, (3) tabela de ultimas interacoes com score de alinhamento, outcome e modelo utilizado, (4) botao de "drill-down" que exibe o prompt completo e os documentos RAG injetados para qualquer interacao especifica, e (5) painel de alertas com recomendacoes automaticas de acao ("Tom desalinhado detectado: considere re-treinar o

modelo com dados recentes").

8. Model Drift Detection e Zehla Guardian

O drift de modelo e um dos maiores riscos em sistemas de ML em producao. Conforme o comportamento dos clientes muda (novas formas de perguntar sobre precos, novos topicos de interesse, mudancas sazonais de demanda), o modelo treinado com dados antigos gradualmente perde eficacia. O ZEHLA ML Brain implementa um sistema de deteccao e remediacao de drift chamado Zehla Guardian, que opera em tres niveis de resposta automatica.

8.1 Deteccao de Drift

O Zehla Guardian monitora tres tipos de drift simultaneamente. O **Data Drift** detecta mudancas na distribuicao das mensagens recebidas (novos topicos, vocabulario diferente, padroes de horario alterados) usando a metrica de Population Stability Index (PSI) calculada semanalmente sobre os embeddings das mensagens. Se o PSI exceder 0.2, o sistema sinaliza drift moderado. Se exceder 0.5, drift severo. O **Concept Drift** detecta mudancas na relacao entre features e outcomes (por exemplo, uma estrategia que funcionava no verao para de funcionar no inverno) monitorando a divergencia entre predicoes do modelo e outcomes reais. O **Performance Drift** detecta degradacao nas metricas de negocio (taxa de conversao, tempo de resposta, satisfacao do cliente) comparando as medias moveis de 7 dias com as medias dos 30 dias anteriores.

8.2 Niveis de Resposta do Guardian

Nivel	Condicao	Acao Automatica	Notificacao
Verde (Normal)	Todas as metricas dentro do threshold	Nenhuma. Operacao normal.	Nenhuma
Amarelo (Atencao)	1 metrica fora do threshold por 24h	Log de warning. Coleta de dados intensificada.	Email ao suporte ZEHLA
Laranja (Alerta)	2+ metricas fora do threshold por 48h	Fallback para modelo base. Resposta final mais conservadora.	Push + Email ao proprietario
Vermelho (Critico)	Tone alignment < 0.3 ou drift severo (PSI>0.5)	Agente desativado. 100% das conversas transferidas para humano.	Push + Email + SMS ao proprietario

Tabela 7: Niveis de resposta do Zehla Guardian

O Zehla Guardian tambem implementa um mecanismo de auto-recovery. Quando um drift e detectado no nivel amarelo, o sistema automaticamente agenda um ciclo de re-treinamento com os dados mais recentes (ultima 2 semanas). Se o re-treinamento resolver o drift, o modelo atualizado e promovido para producao sem intervencao humana. Se o drift persistir apos o re-treinamento, o sistema escala para o nivel laranja e solicita revisao manual da equipe ZEHLA. Este design garante que a maioria dos drifts seja corrigida automaticamente, reservando a intervencao humana apenas para casos que

realmente necessitam de análise especializada.

9. Isolamento Multi-Tenant

O ZEHLA ML Brain opera em um ambiente multi-tenant onde cada pousada é um tenant independente com seus próprios dados, modelos e configurações de ML. O isolamento rigoroso entre tenants é um requisito não negociável, tanto por questões de privacidade (LGPD) quanto por questões de qualidade (o tom de voz da pousada A nunca deve influenciar as respostas da pousada B). O isolamento é implementado em quatro camadas: dados, embeddings, modelos e prompts.

Na camada de **Dados**, todas as queries ao banco de dados incluem obrigatoriamente o filtro `tenantId`. O Prisma middleware (`tenantGuard`) injeta automaticamente este filtro em todas as operações, impedindo que um tenant acesse dados de outro. Na camada de **Embeddings**, cada tenant possui seu próprio namespace no PGVector. Os embeddings são armazenados com metadados que incluem o `tenantId`, e toda query de busca inclui um filtro de metadados que restringe os resultados ao tenant correto. Na camada de **Modelos**, cada pousada com fine-tuning habilitado possui seu próprio modelo dedicado com ID único (`ft-pousada-{tenantId}`). O LLM Router seleciona automaticamente o modelo correto com base no `tenantId` da conversa ativa. Na camada de **Prompts**, cada tenant possui seu próprio system prompt armazenado no banco de dados, que é injetado pelo LLM Router no início de cada conversa. O prompt é cacheado em Redis com TTL de 5 minutos para performance.

Regra de Ouro Multi-Tenant: Nunca consultar embeddings, modelos ou prompts sem o filtro `tenantId`. O `tenantGuard` middleware no Prisma garante isso automaticamente em nível de banco de dados, mas todo acesso direto ao PGVector ou a modelos fine-tuned também deve incluir a verificação explícita.

10. Privacidade e LGPD no Contexto ML

O processamento de Machine Learning envolve grandes volumes de dados de comunicação, o que torna a conformidade com a LGPD particularmente crítica nesta camada do sistema. O ZEHLA ML Brain implementa um framework de privacidade em cinco pilares que garante que o aprendizado do modelo não comprometa os direitos dos titulares dos dados.

10.1 Cinco Pilares de Privacidade ML

- **Consentimento Específico para ML:** O termo de consentimento do proprietário inclui uma cláusula explícita e separada autorizando o uso de suas conversas para treinamento de IA. Sem este consentimento, o ML Brain opera apenas com RAG e não executa fine-tuning.
- **Anonimização Pre-Vetorização:** Antes de gerar embeddings, toda mensagem passa pela pipeline de PII detection e substituição. Embeddings são criados sobre textos anonimizados, nunca sobre dados brutos. O mapeamento PII para anonimizado é mantido apenas por 72 horas para debug, depois descartado de forma segura.

- **Direito ao Esquecimento ML:** Se um proprietario solicita a exclusao de seus dados (LGPD Art. 18), o sistema exclui nao apenas os dados no banco, mas tambem: (a) remove seus embeddings do PGVector, (b) exclui o modelo fine-tuned (se existir) via API, e (c) invalida os caches de prompt no Redis. O processo e concluido em ate 24 horas.
- **Minimizacao de Dados:** O sistema retém apenas os dados minimamente necessarios para o ML. Mensagens com outcome=IGNORED sao descartadas apos 30 dias. Embeddings de conversas com mais de 6 meses e sem conversao associada sao arquivados em cold storage e removidos do index de busca ativo.
- **Audit Trail Completo:** Cada operacao de ML (geracao de embedding, fine-tuning, atualizacao de perfil de voz) e registrada no audit log com: operador (sistema ou usuario), tenantId, tipo de dado processado, timestamp e hash de verificacao. Este log e mantido por 24 meses e disponivel para auditoria.

10.2 Pipeline de Sanitizacao de Dados

A pipeline de sanitizacao opera como um middleware entre a ingestao de mensagens e o processamento de ML. Ela utiliza uma combinacao de regex patterns e NER (via spaCy para portugues) para detectar e substituir dados pessoais. Os padroes cobertos incluem: nomes proprios (detected pelo NER com threshold de confianca 0.85), numeros de telefone brasileiros (regex para formatos +55 XX XXXXX-XXXX e variantes), CPF (regex com validacao de digitos verificadores), emails (regex RFC 5322), enderecos (NER + regex de CEP), dados bancarios (agencia, conta, cartao via regex especificas), e placa de veiculo (regex para formato mercosul e antigo). Cada substituicao e registrada no audit trail com o tipo de dado detectado, a posicao no texto e o hash do original para possivel verificacao futura.

11. Comandos do Agente de Desenvolvimento

Esta secao fornece os comandos e procedimentos que o agente de desenvolvimento deve seguir ao implementar, modificar ou debugar componentes do ML Brain. Todos os comandos assume que o projeto esta na raiz do workspace e utilizam as ferramentas padrao do stack (BullMQ, Prisma, Redis, PGVector).

11.1 Implementacao do Feedback Loop

Ao modificar llm-router.ts ou qualquer componente de IA, o agente deve: (1) Verificar que toda resposta do LLM dispara o registro no MLInteractionLog antes de ser enviada ao WhatsApp, (2) Garantir que o campo toneAlignmentScore e calculado em tempo real comparando o embedding da resposta gerada com o embedding do perfil de voz da pousada, (3) Confirmar que o outcome da conversa e atualizado por webhook quando uma reserva e criada/cancelada, e (4) Validar que o tenantGuard middleware esta ativo em todas as operacoes de banco relacionadas a ML.

11.2 Exportacao para Fine-Tuning

O script de exportacao para fine-tuning esta localizado em scripts/ml-training/export-training-data.ts e deve ser executado semanalmente via cron job. O script seleciona automaticamente conversas com outcome=BOOKED e toneAlignmentScore >= 0.7, formata no padrao jsonl da OpenAI, valida a integridade dos dados e faz upload para um bucket seguro. O agente pode tambem executar manualmente para pousadas especificas passando o tenantId como argumento. O script gera um relatorio de qualidade que inclui: numero de exemplos exportados, distribuicao de sentimento, cobertura de topicos e estimativa de qualidade do dataset.

11.3 Atualizacao do CognitiveObservability

Ao adicionar novas metricas ao ML Brain, o agente deve: (1) Adicionar a metrica ao schema MLInteractionLog se necessario, (2) Atualizar o endpoint /api/ml/metrics para incluir a nova metrica, (3) Adicionar o card correspondente no componente CognitiveObservability.tsx com grafico e thresholds de alerta, e (4) Documentar a metrica no ZCC com descricao, formula de calculo e acao recomendada quando fora do threshold.

11.4 Scripts ML Disponiveis

Script	Localizacao	Funcao	Frequencia
export-training-data.ts	scripts/ml-training/	Exporta conversas para fine-tuning	Semanal (cron)
generate-voice-profile.ts	scripts/ml-training/	Gera Voice Profile a partir de conversas	On-demand
run-fine-tuning.ts	scripts/ml-training/	Submete job de fine-tuning a API OpenAI	Semanal (cron)
evaluate-model.ts	scripts/ml-training/	Avalia modelo fine-tuned vs base	Apos treinamento
detect-drift.ts	scripts/ml/	Calcula PSI e detecta data/concept drift	Diario (cron)
cleanup-old-embeddings.ts	scripts/ml/	Remove embeddings com mais de 6 meses	Mensal (cron)
sanitization-pipeline.ts	scripts/ml/	Executa pipeline de PII detection	Real-time (middleware)

Tabela 8: Scripts ML disponiveis para o agente

12. Roadmap de Implementacao

A implementacao do ZEHLA ML Brain segue um roadmap de 16 semanas dividido em quatro fases, cada uma com entregaveis claros e criterios de aceitacao. As fases sao progressivas: cada uma depende da anterior e adiciona camadas de complexidade e inteligencia ao sistema. O design permite que a primeira fase entregue valor imediato (RAG basico) enquanto as fases subsequentes incrementam

progressivamente a personalizacao e eficiencia.

Fase	Semanas	Entregavel Principal	Criterio de Aceitacao
Fase 1: RAG Foundation	1-4	Pipeline de embeddings + busca semantica	Busca sub-200ms, top-3 relevantes
Fase 2: Voice Cloning	5-8	Pipeline de analise de conversas + Voice Profile	Perfil gerado com 6 dimensoes
Fase 3: Fine-Tuning	9-12	Pipeline de treinamento + deploy de modelos	Modelo fine-tuned supera base em 15%
Fase 4: Guardian	13-16	Drift detection + auto-remediation + observability	Drift detectado em < 24h, recovery automatico

Tabela 9: Roadmap de implementacao do ML Brain

12.1 Fase 1: RAG Foundation (Semanas 1-4)

- **Semana 1:** Setup do PGVector, configuracao do schema de embeddings, implementacao do middleware de sanitizacao PII, e criacao do service de embedding (OpenAI API wrapper com retry e rate limiting).
- **Semana 2:** Implementacao da pipeline de ingestao de conversas (Capture + Enrich), parser de arquivos exportados do WhatsApp (.txt), e integracao com o event pipeline existente (Phase 4 Brain Sync).
- **Semana 3:** Implementacao da busca semantica (Retrieve) com reranker, integracao com o llm-router para injecao de contexto RAG no prompt, e testes de qualidade de busca com 50 cenarios simulados.
- **Semana 4:** Implementacao do MLInteractionLog basico (sem fine-tuning), metricas de Tone Alignment Score, e deploy em staging para testes com 3 pousadas piloto.

12.2 Fase 2: Voice Cloning (Semanas 5-8)

- **Semana 5:** Implementacao completa do Voice Fingerprinting (6 dimensoes), parser avancado de conversas WhatsApp com deteccao de turnos, e calculo do Indice de Formalidade.
- **Semana 6:** Analise de sentimento por turno (BERTimbau), extracao de topicos com LDA, e deteccao de estrategias de conversao correlacionadas com outcome=BOOKED.
- **Semana 7:** Geracao automatica do Voice Profile Document (JSON estruturado), integracao com o DNA Wizard (preenchimento automatico do Tone Thermometer), e fluxo de validacao pelo proprietario.
- **Semana 8:** Teste de Turing Parcial (10 cenarios simulados para validacao pelo proprietario), refinamentos baseados em feedback dos pilotos, e documentacao completa.

12.3 Fase 3: Fine-Tuning (Semanas 9-12)

- **Semana 9:** Implementacao do script de exportacao para fine-tuning (selecao, formatacao, validacao), integracao com a API de Fine-Tuning da OpenAI, e pipeline de submissao de jobs.
- **Semana 10:** Implementacao do script de avaliacao de modelos (base vs fine-tuned em 50 cenarios), pipeline de promocao para producao (A/B testing), e rollback automatico em caso de regressao.
- **Semana 11:** Implementacao do LLM Router com selecao automatica de modelo (base vs fine-tuned por tenant), cache de modelos no Redis, e fallback graceful para modelo base em caso de falha.
- **Semana 12:** Cron job semanal de treinamento automatico, otimizacao de custos (compressao de prompts, cache de embeddings), e deploy em producao para todas as pousadas ativas.

12.4 Fase 4: Guardian (Semanas 13-16)

- **Semana 13:** Implementacao do calculo de PSI para deteccao de data drift, deteccao de concept drift (prediction vs outcome), e deteccao de performance drift (metricas de negocio).
- **Semana 14:** Implementacao dos quatro niveis de resposta do Zehla Guardian (verde/amarelo/laranja/vermelho), sistema de auto-recovery com re-treinamento automatico, e notificacoes multi-canal.
- **Semana 15:** Implementacao completa do CognitiveObservability.tsx no ZCC (6 metricas em tempo real, graficos interativos, drill-down de interacoes, painel de alertas).
- **Semana 16:** Testes de integracao end-to-end, testes de stress (1000 conversas simultaneas), audit de seguranca LGPD, e documentacao final de producao.